

Разработка AI-сервиса техподдержки

Выполнил:

студент 4 курса

09.03.01 Информатика и вычислительная
техника, Технологии разработки программного
обеспечения

Гневнов Артем Евгеньевич

Руководитель:

кандидат физико-математических наук, доцент
кафедры ИТиЭО Жуков Николай Николаевич

Актуальность

Потребность: локальный AI-сервис, встраиваемый в систему управления заявками организации, без передачи данных внешним провайдерам

Предмет и объект

ОБЪЕКТ ИССЛЕДОВАНИЯ

Процесс автоматизированной обработки обращений пользователей в системе технической поддержки.

ПРЕДМЕТ ИССЛЕДОВАНИЯ

Программные методы разработки AI-сервиса технической поддержки в микросервисной архитектуре с применением локальных языковых моделей.

Цель

Разработать микросервис AI-технической поддержки, обеспечивающий:

- автоматическую обработку пользовательских обращений на основе локальной языковой модели;
- сохранение полной истории диалога;
- ограничение нагрузки на языковую модель;
- интеграцию в систему управления заявками.

Задачи

1. Сравнительный анализ существующих AI-решений в области автоматизации технической поддержки.
2. Формулировка функциональных и нефункциональных требований к разрабатываемому AI-сервису.
3. Проектирование архитектуры микросервиса helperbot-ai и протокола его интеграции с системой управления заявками.
4. Реализация микросервиса helperbot-ai: gRPC-интерфейс, хранение чатов в PostgreSQL, вызов локальной LLM через Ollama.
5. Интеграция helperbot-ai с сервисом бизнес-логики helperbot-core и Telegram-ботом helperbot-tg.
6. Тестирование системы и описание процедуры развёртывания через Docker Compose.

Инструменты и технологии

Компонент	Технология	Обоснование выбора
Язык бэкенда	Go 1.24+	Горутины, статический бинарник, нативный gRPC
Протокол	gRPC + Protocol Buffers	Типизированные контракты, HTTP/2
Валидация	protoc-gen-validate	Декларативные правила валидации в .proto-файле
СУБД	PostgreSQL 16	ACID, UUID, гибкое индексирование
Драйвер БД	pgx v5 + pgxpool	Нативный Go-драйвер, пул соединений
DI-фреймворк	Uber FX	Декларативное внедрение зависимостей
LLM-среда	Ollama	Локальный запуск, HTTP API, Modelfile
Языковая модель	qwen2.5:1.5b	Компактность, поддержка русского языка
Миграции	Goose v3	SQL-миграции, версионирование, Docker
Оркестрация	Docker Compose	Единый запуск всего комплекса

Результат

1. Анализ — сравнение 5 решений → обоснован выбор Ollama.
2. Требования — 6 функциональных + 8 нефункциональных, закреплены в protoc-gen-validate.
3. Архитектура — gRPC-контракт из 2 методов, БД из 2 таблиц, без сущностей системы-хоста.
4. Реализация — Go + Uber FX; rate limit на SQL; модель support-bot (qwen2.5:1.5b).
5. Интеграция — прокси в helperbot-core, FSM в helperbot-tg; типизированные ошибки (errors.Is).
6. Тесты и развёртывание — 10 юнит-тестов; запуск одной командой make compose-up.

Демонстрация работы продукта

